Viva preparation: RL for Sequential Recommendation

Concepts, implementation details, and likely cross-questions

- 1. The 60-second pitch
- 2. Core concepts from first principles
  - 2.1 The MDP for recommendation
  - 2.2 MNL choice model and the outside option
  - 2.3 Combinatorial (assortment) MDP and $Q(s, A) = \sum_a P(a \mid s, A)\bar{Q}(s, a)$
  - 2.4 Assortment optimisation: Charnes-Cooper LP and exact bisection
  - 2.5 Optimism and pessimism in MNL-VQL, in simple words
  - 2.6 SQN, EVAL and HIST heads, and Labarca et al.'s finding
  - 2.7 DQN essentials: target networks, double DQN, Polyak averaging
  - 2.8 Off-policy and offline RL, distribution shift, CQL
  - 2.9 HR@k and NDCG@k
  - 2.10 The theory results with intuitive proofs
- 3. How each step was implemented (S1-S9)
  - S1-S2: Data
  - S3-S4: Labarca et al. reproduction
  - S5: Lee & Oh synthetic experiment
  - S6: Formulation (`report/formulation.tex`)
  - S7: MovieLens-1M simulator (`rlrec/sim/user_sim.py`)
  - S8: Learned agents in the simulator ($K = 1$) (`rlrec/agents/sim_agents.py`, `scripts/run_sim_m1.py`)
  - S9: Demo (`app/demo.py`, `scripts/train_demo_models.py`)
- 4. Likely cross-questions with answers
- 5. Honest limitations, and what pre-compre will add

> How to use this sheet: each concept is first explained in plain words (what you say aloud), then in math (what you write on the board). File and function names show where something lives in the code, so if the professor asks "show me", you can open it. Numbers marked **"running on GPU; to be added"** are still running; say so openly instead of guessing. S8 numbers are the final 3-seed means (`report/build/midsem_report.md`, section 6).

1. The 60-second pitch

Question. A supervised next-click recommender is trained to predict, and show, what the user will click *right now*. An RL recommender is trained to maximise *total* engagement over the session. When does the RL recommender actually win on long-term utility, and how does showing a slate of K items (re-ranking) change the answer?

Framing (our main conceptual contribution). We put both recommenders inside one model: an episodic **combinatorial MDP with multinomial-logit (MNL) user choice** (Lee & Oh, ICML 2025). In it:

- the supervised recommender is exactly the **myopic, $\gamma = 0$ policy** (Lemma 1);
- RL is the $\gamma > 0$ policy;
- the slate size K is an explicit parameter ($K = 1$ is sequential recommendation, $K > 1$ is re-ranking).

We proved two bounds on when RL can help: the myopic policy can be **arbitrarily worse** than optimal on long-term return (Prop. 1), but RL has **no advantage at all** when what the user picks does not change the user's future state (Prop. 2).

What we built.

1. A reproduction of Labarca et al. (ICML 2024) on RetailRocket: GRU4Rec and SASRec with CE, SQN, EVAL and HIST heads (S3-S4).
2. A reproduction of Lee & Oh's synthetic experiment: MNL-VQL vs LSVI-UCB vs Myopic, with an exact assortment solver (Charnes-Cooper LP and bisection) (S5).
3. A MovieLens-1M user simulator with satisfaction, genre fatigue, clickbait "junk" items and exit, plus a strength knob s that switches long-term effects on and off (S7).
4. Agents in that simulator: supervised CE, SQN-style, CQL, our MNL-Q, candidate-restricted DQN, heuristics and oracles (S8), and a Streamlit demo (S9).

Main finding so far.

- Synthetic (Lee & Oh setting): MNL-VQL reaches about 99.7% of the optimal return, while Myopic sits 25-37% below it.
- Simulator, oracles: with no long-term effects ($s = 0$) the myopic oracle wins (6.19 vs 4.95 clicks per session). With strong effects and slates ($s = 1$, $K = 5$), the quality oracle gets **2.5x** more clicks per session (6.03 vs 2.40), even though the myopic oracle always has the higher first-slate CTR.
- Simulator, learned agents ($K = 1$, 3 seeds): MNL-Q's advantage over supervised grows with the strength s: -4% at $s = 0$ (3.89 vs 4.06), $+1\%$ at 0.25, $+5\%$ at 0.5, $+11\%$ at $s = 1$ (1.31 vs 1.18), with 43% fewer junk clicks (0.33 vs 0.58 per session). That is the crossover the theory predicts. However, a simple non-learned "highest estimated rating" heuristic is still better for $s > 0$ (1.62 at $s = 1$), and the quality oracle reaches 2.03. Learned RL beats supervised only in the regime where it should, and closing the gap to the oracle is the remaining work.

2. Core concepts from first principles

2.1 The MDP for recommendation

Plain words. The recommender is the agent and the user is the environment. At each step the agent shows something, the user reacts (clicks one item or nothing), the agent gets a reward (1 for a click), and the user's internal state changes (they get more or less satisfied, more tired of a genre, or they leave).

Math (formulation.tex, Def. 1). $\mathcal{M} = (\mathcal{S}, \mathcal{I}, \mathcal{A}_K, P_h, p_h, r_h, H)$:

Element	Meaning in general	In our simulator (`rlrec/sim/user_sim.py`)
state s	what describes the user now	observed: user id, last 10 clicked items, step t. Hidden: satisfaction $\sigma \in [0,1]$, genre fatigue $f \in \mathbb{R}^{18}$
action A	a slate of $\le K$ items, plus the outside option a_0	K item ids out of 500 movies
choice $P(a \mid s, A)$	which item the user picks	MNL (below)
reward r	immediate payoff	1 per click, 0 for no click
transition $p(s' \mid s, a)$	how the user changes	satisfaction update, fatigue update, junk-induced exit
horizon H	session cap	$H = 30$
discount γ	weight of future reward	agents use $\gamma = 0.9$; long-term metric is undiscounted clicks per session

What γ means for a recommender. γ says how much a click t steps from now is worth compared with a click now (γ^t). $\gamma = 0$ means "only this click matters", which is the CTR model. $\gamma \to 1$ means

"every click in the session counts equally", which is clicks per session. With exit, γ also works as an effective horizon of about $1/(1-\gamma)$ steps (10 steps for 0.9).

Why it matters here. The whole project is the comparison $\gamma = 0$ vs $\gamma > 0$ in the same model. Without a transition that depends on the action, no γ changes the optimal policy (Prop. 2).

2.2 MNL choice model and the outside option

Plain words. Each item has an "attractiveness" score (utility). The user picks an item with probability proportional to e^{utility}. "Not clicking anything" is also an option, with utility 0. So showing more attractive items lowers the chance of no click.

$$P(a \mid s, A) = \frac{e^{v(s,a)}}{1 + \sum_{a' \in A} e^{v(s,a')}}, \qquad P(a_0 \mid s, A) = \frac{1}{1 + \sum_{a' \in A} e^{v(s,a')}}.$$

The "1" in the denominator is e^0, the outside option.

Simulator utility (`UserSim.utilities`, `base_util`):

$$v(u,i) = b_0 + \beta_r(\hat{r}_{ui} - 3.5) + \kappa\,\mathbb{1}[\text{junk}_i] - s\,\lambda_f\,\langle f, g_i\rangle,$$

with $b_0 = -2$, $\beta_r = 1$, $\kappa = 1$ (junk is *more* clickable), $\lambda_f = 1.5$, g_i = normalised genre vector.

Properties to know.

- **IIA (independence of irrelevant alternatives):** the ratio $P(a)/P(b)$ does not depend on what else is shown. This is what makes slate optimisation tractable.
- With $K = 1$: $P(\text{click } a \mid \{a\}) = \sigma(v) = e^v/(1+e^v)$. MNL-Q uses exactly this (`mnlq_scores`: `torch.sigmoid(out["choice"])`).
- Training loss `choice_nll` (in `sim_agents.py`) is a cross-entropy over $\{a_0\} \cup A$ with a_0's logit fixed at 0. Because the outside option is in the loss, the model learns *absolute*, calibrated utilities and not only a ranking. An ordinary softmax over items cannot do that, since it is shift-invariant.

2.3 Combinatorial (assortment) MDP and $Q(s,A) = \sum_a P(a \mid s, A)\,\bar{Q}(s,a)$

Plain words. The user picks only one item from the slate, so the value of a slate is the average of the values of "what happens if the user picks a", weighted by how likely each pick is. We therefore only need to learn values for single items ($N+1$ numbers per state), not for every possible slate. There are $\sum_{m \le K} \binom{N}{m}$ slates: 637 for $N = 10$, $K = 5$, and 760,098 for $N = 40$.

$$\bar{Q}_h(s,a) = r_h(s,a) + \gamma\,\mathbb{E}_{s' \sim p_h(\cdot \mid s,a)} V_{h+1}(s'), \qquad Q_h(s,A) = \sum_{a \in A \cup \{a_0\}} P_h(a \mid s, A)\,\bar{Q}_h(s,a), \qquad V_h(s) = \max_A Q_h(s,A).$$

Why it matters. This is why MNL-VQL scales and LSVI-UCB (which treats each slate as an atomic action) does not. It is also the structure of our MNL-Q: $Q(s,\{a\}) = p\,Q_c(s,a) + (1-p)\,N(s)$, where $N(s) = \bar{Q}(s,a_0)$.

2.4 Assortment optimisation: Charnes-Cooper LP and exact bisection

The problem (`rlrec/slate/assortment.py`). Given weights $w_a = e^{v_a}$, item values f_a, and the outside option (w_0, f_0):

$$\max_{A,\ 1 \le |A| \le K} F(A) = \frac{w_0 f_0 + \sum_{a \in A} w_a f_a}{w_0 + \sum_{a \in A} w_a}.$$

This is a ratio of two linear functions of the 0/1 inclusion vector x.

Why the LP works (`solve_lp`).

1. *Relaxation is exact.* Replace $x_a \in \{0,1\}$ by $0 \le x_a \le 1$. For MNL with a cardinality constraint, the relaxed problem has an optimal solution that is integral (Davis, Gallego & Topaloglu 2013, cited by Lee & Oh, App. C). The constraint matrix after the transformation has the right (totally-unimodular-like) structure, so a vertex solution is 0/1.
2. *Charnes-Cooper* turns a linear-fractional program into an LP. Let $t = 1/(w_0 + \sum_a x_a w_a)$ and $y_a = x_a t$. Then

$$\max_{y,t} \ \sum_a f_a w_a y_a + f_0 w_0 t \quad \text{s.t.} \quad \sum_a w_a y_a + w_0 t = 1, \ \ \sum_a y_a \le K t, \ \ 0 \le y_a \le t.$$

   Recover $x_a = y_a/t$. The code takes $x_a > 0.5$. If the LP prefers the empty set (when f_0 dominates), it falls back to the best single item, because our slates must contain at least one item.

Why bisection is exact (`solve_parametric`). Key identity: since the denominator is positive,

$$F(A) \ge \lambda \ \Leftrightarrow \ w_0(f_0 - \lambda) + \sum_{a \in A} w_a(f_a - \lambda) \ge 0.$$

Define $g(\lambda) = \max_A \left[w_0(f_0 - \lambda) + \sum_{a \in A} w_a(f_a - \lambda) \right]$.

- For a *fixed* λ the inner max is separable: take the top-K positive terms $w_a(f_a - \lambda)$, and always at least the single best term because of the $|A| \ge 1$ constraint.
- g is continuous and strictly decreasing in λ, and $g(\lambda) \ge 0 \ \Leftrightarrow \ \lambda \le \max_A F(A)$. So the optimum λ^* is the root of g, and bisection finds it.
- 60 iterations shrink the interval by a factor of 2^{60}, which is machine precision. The set chosen at the final lower end achieves λ^*.
- It is vectorised over all states at once, which is why we use it inside planning. The LP is kept for verification and for the planned neural re-ranker.
- Both solvers are checked against brute-force enumeration in `tests/test_assortment.py` (`test_solvers_match_bruteforce`).

Intuition for the answer. The optimal slate is "all items whose value beats the slate's own average value, up to K of them". Adding an item helps only if its value is above the current ratio.

2.5 Optimism and pessimism in MNL-VQL, in simple words

Optimism in the face of uncertainty. When unsure, act as if things you have tried little might be great. You either get the reward or learn they are not, so you explore automatically. UCB = estimate + bonus.

MNL-VQL has two unknowns: user preferences (MNL parameter θ) and long-term item values ($\bar{Q}$).

1. **Optimistic item values** $f_{h,1} = \hat{f} + b$ (ridge regression on linear-MDP features $\psi(s,a)$ plus the elliptical bonus $b = \beta \, \| \psi \|_{\Lambda^{-1}}$).
2. **The subtle part (Eq. 7).** In MNL *bandits* item values are known and positive, and the outside option is worth 0. Making utilities optimistic (higher) then always helps, because it pulls probability away from "no click". Here the outside option has a *non-zero, unknown* value (not buying raises the budget). If the outside option is the best thing, making items look more attractive would *reduce* the slate's value, which is not optimistic. So: **if some item's optimistic value beats the outside option's, use optimistic utilities; otherwise use pessimistic utilities.** Either way, the estimate errs on the side of a higher slate value. Code: `_feasible_utilities` in `rlrec/agents/combinatorial.py`.
3. **"Overly optimistic" and "overly pessimistic"** estimates $f_{h,2} = \hat{f} + 3b$ and $f_{h,-2} = \hat{f} - b$ are for the theory (variance-weighted regression). Eq. 9: if $f_{h,1}$ and $f_{h,2}$ disagree by more than a threshold u on the planned slate, switch to the $f_{h,2}$ slate for the rest of the episode (`MNLVQL.act`, `u_switch = 0.5`).

4. **MNL estimation by online mirror descent** (`MNLEstimator.update`): one Newton-like step per observation, at constant cost per step, with H_t accumulating the Fisher information. The confidence width is $\alpha\sqrt{\log(2+t)}\ \|\phi\|_{H^{-1}}$.

Guarantee (Lee & Oh Thm 5.1/5.2). Regret grows like $\sqrt{K}$ in the number of episodes K (leading terms of order $d\sqrt{HK}$ from the MNL model and $d_{lin}H\sqrt{K}$-type terms from the Q-values), and for linear MDPs it is nearly minimax-optimal, with a matching lower bound $\Omega(d\sqrt{HK} + d_{lin}\sqrt{HK})$ in Thm 5.3. Say "nearly minimax optimal in the linear MDP case" and do not try to recite the exact constants.

2.6 SQN, EVAL and HIST heads, and Labarca et al.'s finding

Setting. Next-item prediction (NIP) on logged sessions. A backbone (GRU4Rec / SASRec) maps the last 10 items to a hidden vector z. The **action-logits head** is trained with cross-entropy to predict the next item, and **recommendations always come from this head**. An **auxiliary head** on the same z adds a second loss:

Head	Loss on the auxiliary head (one output per item)	Code
none	–	`SeqRecModel(head="none")`
SQN (Xin et al. 2020)	$(r + \gamma \bar{Q}'(s', \arg\max_{a'} Q(s', a')) - Q(s, a))^2$, double DQN	`losses`, `head=="sqn"`
EVAL (Labarca)	SARSA / policy evaluation: $(r + \gamma Q'(s', a'_{\text{logged}}) - Q(s, a))^2$	`head=="eval"`
HIST (Labarca)	$(Q(s, a) - \min(\#\text{past interactions}, 50))^2$, plain regression	`head=="hist"`

Rewards: click 1, purchase (add-to-cart) 5, $\gamma = 0.5$ (`rlrec/data.py`, `CLICK_REWARD, BUY_REWARD`).

Labarca et al.'s argument.

1. *Theory (Thm 4.3):* for any consistent NIP metric (HR, NDCG) and $\gamma > 0$, an optimal RL policy can be arbitrarily worse than the CE solution on NIP. *Thm 4.4:* the CE optimum is at least as good as any optimal policy on NIP. So "RL improves NIP" cannot be because it is doing RL.
2. *Top-down:* with $\gamma = 0.5$ and rewards 1/5, Q is dominated by the next 2-3 rewards. Learning Q is therefore like predicting "will the next items be clicks or buys". EVAL (no max, no control) performs as well as SQN.
3. *Bottom-up:* feature-importance (SHAP) on SQN's Q-values shows the top explanatory feature is **hist-length**, the number of interactions so far in the session.
4. *HIST*, a plain regression of that count, matches SQN. **Conclusion: the gain comes from an auxiliary loss shaping the representation, not from RL's long-term optimisation.**

Why it matters here. It motivates our pivot. If you want to show RL helps *long-term utility*, NIP on a static log cannot show it (it rewards $\gamma = 0$ by construction). You need an environment where actions change the future, hence the simulator.

2.7 DQN essentials: target networks, double DQN, Polyak averaging

- **Q-learning target:** $y = r + \gamma \max_{a'} Q(s', a')$. Regressing $Q(s, a)$ on a target that uses the same network chases a moving target and can diverge.
- **Target network** $Q_{\theta'}$: a slow copy used to compute y, which stabilises training.
- **Polyak averaging:** $\theta' \leftarrow (1 - \tau)\theta' + \tau\theta$ after every update, instead of a hard copy every C steps. We use $\tau = 0.01$ (RetailRocket SQN and offline sim agents) and $\tau = 0.005$ (online DQN).

- **Overestimation:** $\mathbb{E}[\max_a \hat{Q}] \geq \max_a \mathbb{E}[\hat{Q}]$, and noise in many actions inflates the max. **Double DQN:** choose $a' = \arg\max_a Q_\theta(s', a)$ with the online net, and evaluate it with the target net $Q_{\theta'}(s', a')$, which decorrelates selection and evaluation errors. Used in `losses` (sqn), `train_offline`, and `train_dqn`.
- **Why plain DQN failed over 500 items:** see Q&A. In short, the max over 500 noisy estimates is dominated by noise, and exploration over 500 items wastes most interactions on irrelevant items.

2.8 Off-policy and offline RL, distribution shift, CQL

- **On-policy:** learn from data generated by the current policy. **Off-policy:** learn about a target policy from data generated by another (behaviour) policy, e.g. DQN with a replay buffer. **Offline (batch) RL:** a fixed log, with no further interaction.
- **Distribution shift / extrapolation error.** The learned greedy policy picks actions the logger rarely showed. Their Q values are pure extrapolation, often over-estimated, and the $\max$ in the target propagates those errors. Offline, nothing corrects them.
- **CQL (Kumar et al. 2020):** add $\alpha\, \mathbb{E}_s[\log \sum_a e^{Q(s,a)} - Q(s, a_{\text{data}})]$. This pushes down Q on all actions (soft-max) and pushes up Q on logged actions, giving a lower bound on the policy's value. Code: `train_offline(kind="cql")`, $\alpha = 1$. In our results CQL is conservative: low junk clicks but also low CTR.
- **Our logs** (`collect_logs`): a behaviour policy that with probability 0.7 samples from a softmax over popularity rank (temperature 50 ranks) and with probability 0.3 samples uniformly; 2 sessions per training user.

2.9 HR@k and NDCG@k

There is one true next item per prediction. Let rank be its 0-based position in the full ranking over all items (`nip_metrics.evaluate`: number of items with a strictly larger logit; padding item masked).

$$\text{HR@}k = \mathbb{1}[\text{rank} < k], \qquad \text{NDCG@}k = \frac{\mathbb{1}[\text{rank} < k]}{\log_2(\text{rank} + 2)}.$$

With a single relevant item the ideal DCG is 1, so no normalisation term is needed. NDCG gives 1 at rank 0, 0.63 at rank 1, 0.5 at rank 2, and so on. Both metrics are reported separately for click targets and purchase targets, averaged over every event in the test sessions, with **full ranking** (no sampled negatives). Ties count in the target's favour (strict $>$), which is negligible with float logits.

2.10 The theory results with intuitive proofs

Lemma 1 (supervised top-K = myopic). *Plain words:* if every click is worth 1, the slate that maximises the chance of *some* click is the K most attractive items. So a CE model that learned the true utilities and shows its top K is the $\gamma = 0$ policy. *Proof:* $P(\text{click} \mid A) = W/(1 + W)$ with $W = \sum_{a \in A} e^{v_a}$, which increases in W, and W is maximised by the K largest weights. Softmax is shift-invariant, so CE scores rank like utilities. **Assumptions (the professor may ask):**

1. Equal reward for every click. With buys worth 5, top-K by probability is *not* myopic; you need the revenue-ordered assortment, which is exactly the assortment solver.
2. MNL/IIA choice holds, with no position effects inside the slate.
3. The model recovers the utilities up to a constant. That needs sufficient capacity, enough data, and *no exposure confounding*. A model trained on logged clicks learns $P(\text{click} \mid \text{shown})$ only if it conditions on what was shown, which our `choice_nll` does.
4. The same state information is available to both.

Prop. 1 (myopic can be arbitrarily worse). *Story:* a clickbait item j that is more attractive than a good item g but makes the user leave when clicked. Myopic always shows j, so it gets at most 1 click and then the user is gone. Showing g forever gives Hc_g clicks. Take $H > n/c_g$ and the ratio exceeds n. *Why it matters:* it is the long-term dual of Labarca's Thm 4.3. Neither objective dominates the other; the metric and the dynamics decide.

Prop. 2 (no advantage without choice-dependent dynamics). *Story:* if the user's future does not depend on what they pick, then whatever you show, the future is the same, so just maximise today's reward. *Proof:* $Q^*(s,A) = \sum_a P(a \mid s,A)(r(s,a) + c(s)) = \text{immediate}(A) + c(s)$, since the probabilities (including a_0) sum to 1. The constant $c(s)$ does not affect the argmax. **Important subtlety:** "does not depend on the item chosen" must include a_0. If clicking anything vs nothing changes exit, the dynamics *are* choice-dependent. In our simulator at $s = 0$, exit is a constant `base_exit` and fatigue does not enter utility, so Prop. 2 applies exactly (`test_strength_zero_makes_dynamics_choice_independent`). *Corollary:* the first axis of the regime map is "how much does the choice change the future", which is our knob s.

The slate-size two-type toy model (a conjecture) (formulation.tex Sec. 4; stated as an analysis plan, not a proven result). Be clear about that if asked. Good items (utility v_g, safe) and junk items ($v_j > v_g$, exit probability p on click), with no-click exit p_0. A stationary slate with k junk items:

$$c(k) = \frac{ke^{v_j} + (K-k)e^{v_g}}{1 + ke^{v_j} + (K-k)e^{v_g}}, \quad e(k) = p\frac{ke^{v_j}}{1+\dots} + p_0(1 - c(k)), \quad V(k) = \frac{c(k)}{1 - \gamma(1 - e(k))}.$$

Myopic uses $k = K$; optimal maximises $V(k)$. *Worked illustration* (my numbers: $v_j = 0$, $v_g = -1$, $p_0 = 0.05$, $\gamma = 0.95$, $p = 0.3$): $V(k^*)/V(k^{my}) = 1.37$ ($K = 1$), 2.10 ($K = 2$), 3.44 ($K = 5$), 4.51 ($K = 10$). **Intuition:** with bigger slates the user clicks *something* almost surely either way, so the junk's extra attractiveness buys little CTR but still costs exits. The RL advantage grows with K. This matches S7: at $s = 1$ the quality/myopic ratio is 1.4 at $K = 1$ and 2.5 at $K = 5$.

3. How each step was implemented (S1-S9)

S1-S2: Data

RetailRocket (`data/preprocess_retailrocket.py`):

- views become clicks and add-to-cart becomes purchases (`is_buy`); transaction rows are dropped;
- visitor id = session id (as in the SQN preprocessing);
- sessions and items with ≤ 2 events are removed;
- time-based 80/10/10 split by session start time (Hidasi & Czapp 2023, as Labarca).

Stats: 195,524 sessions, 1,233,949 events, 70,852 items, which match the paper's counts (195,525 / 1,233,949 / 70,852). Train/val/test = 156,419 / 19,552 / 19,553 sessions.

Transitions (`rlrec/data.py:build_transitions`): one per event. The state is the last 10 items (left-padded with 0), the action is the event's item, and the next state appends it. `next_action` is the following item (for EVAL), `hist_len` = t (for HIST), and `done` is set at the last event.

MovieLens-1M (`data/preprocess_ml1m.py`):

- a catalogue of the 500 most-rated movies;
- explicit ALS (rank 32, $\lambda = 0.05$, 8 iterations) on all ratings, which gives $\hat{r}_{ui}$ clipped to [1,5];
- 18 genres, multi-hot;
- **junk = bottom 25% of catalogue items by mean rating**, i.e. popular (they are in the top 500 by count) but poorly rated. This follows Lee & Oh's "provocative title, poor content" junk items.

History features (`data/history_features.py`): a *second* ALS on a random **50%** of ratings. Agents warm-start their user/item embeddings from it. They therefore know something about each user but not the simulator's ground truth (the script reports the mean per-user correlation with $\hat{r}$).
Deviation: datasets came from public GitHub mirrors because Kaggle and GroupLens were unreachable (RetailRocket `events.csv` is byte-identical).

S3-S4: Labarca et al. reproduction

- **Models** (`rlrec/models/seqrec.py`): GRU4Rec (embedding 64, 1-layer GRU, final hidden state) and SASRec (1 block, 1 head, hidden 64, pre-norm, causal mask, dropout 0.1, learned positions). A special case: the all-padding first state is allowed to attend to itself, to avoid NaN attention.
- **Heads:** `SeqRecModel` = logits head + optional aux linear head (one output per item). `aux_at` computes only the needed item's output, to avoid a 70k-wide matmul.
- **Loss:** `CE + aux` (equal weights), with the padding item masked from the CE.
- **Training** (`scripts/train_nip.py`): Adam lr 0.005, batch 256, $\gamma = 0.5$, $\tau = 0.01$, state length 10. Model selection is by best **validation purchase NDCG@20**, and the test metrics of that epoch are reported. Grid: `scripts/run_nip_grid.sh` (2 backbones x 4 heads, `EPOCHS=5`, patience 2, 1 seed).
- **Deviations and why:**
  1. *Compute:* 4-core CPU, about 47-75 min per epoch (GRU-CE epoch 0: 2,839 s; GRU-SQN: 4,311 s). So we used 1 seed and at most 5 epochs instead of 5 seeds and 50 epochs. A Colab GPU run is in progress.
  2. *SQN target:* Polyak target plus double-DQN selection, instead of SQN's two alternately-updated networks. Same purpose (decorrelating the max), simpler.
  3. *HIST target clipped at 50*, to keep the MSE stable on very long sessions.
- **Partial CPU results** (`results/nip_partial_cpu.md`, 1 seed, 2-3 epochs, test purchase metrics):

Model	best epoch	buy HR@20	buy NDCG@20	click HR@20	Paper buy HR@20 / NDCG@20
GRU4Rec (CE)	2 of 3	0.458	0.343	0.292	0.2306 / 0.1450
GRU4Rec-SQN	2 of 2	0.386	0.301	0.249	0.2698 / 0.1743
SASRec (CE)	3 of 3	0.551	0.412	0.346	0.3509 / 0.2176
SASRec-SQN	2 of 2	0.461	0.385	0.267	0.4227 / 0.2634

EVAL and HIST rows and the final multi-epoch numbers: **running on GPU; to be added**. At 2 epochs SQN is *below* CE. The TD loss initially competes with CE for the shared representation (SQN's CE loss is higher at the same epoch: 7.20 vs 6.37 for GRU at epoch 1), so no conclusion yet. - **Absolute numbers are about 2x the paper's.** Measured cause: 67% of test purchase targets are the *same item as the last viewed item* (view, then add-to-cart), and 73% appear somewhere in the 10-item state. For click targets these are 24% / 37%. Repeat consumption makes purchases easy in our preprocessing, and the paper's pipeline likely de-duplicates or handles events differently. **We reproduce the protocol and compare relative CE-vs-head differences, not absolute values.**

S5: Lee & Oh synthetic experiment

- **Environment** (`rlrec/sim/budget_env.py:BudgetEnv`), the "online shopping with budget" MDP from App. G:
  - $|S| = 5$ budget levels, $H = 5$, $M = 6$ (so $K = 5$ items), $d = 5$, start at the middle level;
  - buying a_i in s_j gives reward $(i/(100N) + j/|S|)/H$, and the budget goes up w.p. $1 - i/N$ and down w.p. i/N;
  - buying nothing gives 0 reward and the budget goes up deterministically.

    - The outside option therefore has *long-term value* (it raises the budget), which is exactly why myopic fails: it sells now when waiting would sell at a higher price later.
- **Optimal policy:** exact backward DP with the true model, using the bisection solver at every (h, s) (`optimal_policy`, $\gamma = 1$).
- **Agents** (`rlrec/agents/combinatorial.py`):
    - *Myopic* (OFU-MNL+ adapted): optimistic MNL, empirical immediate reward + bonus $\beta/\sqrt{n+1}$, and $f_0 = 0$.
    - *LSVI-UCB*: tabular over all 637 / 21,699 slates, count-based bonus, unvisited = optimistic 1.0.
    - *MNL-VQL*: OMD MNL estimation, ridge regression on ψ, optimistic / overly optimistic / pessimistic f, Eq. 7 switching, exact assortment, Eq. 9 policy switch.
- **Runner** (`scripts/run_synthetic.py`): 30,000 episodes, 5 seeds, re-plan every 5 episodes, metric = mean return over the last 2,000 episodes. LSVI-UCB is skipped at $N = 40$ (760,098 slates).
- **Deviations:**
    1. $\psi(s, a) = [P(\cdot \mid s, a), r(s, a)]$, the exact linear-MDP features of dimension $|S| + 1 = 6$, instead of the SVD construction. Same span, same dimension.
    2. 5 instances instead of 10.
    3. One MNL θ shared across h (the environment is stationary).
    4. Tuned bonus scales ($\alpha = 0.5$, $\beta = 0.05$, $u = 0.5$; LSVI $\beta = 0.2$) instead of theoretical constants.
    5. Re-plan every 5 episodes (speed).
    6. $\bar{\sigma} = 1$ (unweighted regression), as the paper's own experiments do.
- **Results** (mean over 5 seeds, last 2,000 episodes; % of optimal):

N	Optimal	Myopic	LSVI-UCB	MNL-VQL
10	0.680	0.513 (76%)	0.513 (76%)	0.678 (99.7%)
20	0.719	0.524 (73%)	0.546 (76%)	0.718 (99.8%)
40	0.705	0.444 (63%)	not run	0.706 (100%)

N=10 seed 0: optimal 0.774, MNL-VQL 0.771, LSVI-UCB 0.593, Myopic 0.509. **Be careful:** LSVI-UCB is between the two on average at N=20 and on some seeds, but at N=10 it equals Myopic on average (it is below Myopic on seeds 3 and 4). It has not finished exploring 637 slates x 5 states x 5 steps in 30k episodes. MNL-VQL slightly "above optimal" on some seeds (e.g. 0.731 vs 0.725) is sampling noise in a 2,000-episode mean.

S6: Formulation (`report/formulation.tex`)

This step contains Lemma 1, Props 1-2, and the slate toy model (section 2.10 above).

S7: MovieLens-1M simulator (`rlrec/sim/user_sim.py`)

Batched numpy simulation of B users. Each step:

1. **Choice:** MNL over the slate with utility v above. **Reward** = 1 per click.
2. **Satisfaction** (if click on i): $\sigma \leftarrow \text{clip}(\sigma + s\eta(q_{ui} - q_{ref}))$, with $q_{ui} = (\hat{r}_{ui} - 1)/4$, $\eta = 0.4$, $q_{ref} = 0.6$ (a predicted rating of 3.4). A **junk click** also gives $\sigma - = s \cdot 0.15$ and the user leaves w.p. $s \cdot 0.3$.
3. **Fatigue:** $f \leftarrow 0.8f + g_i$ on click. It lowers the utility of similar-genre items by $s\lambda_f \langle f, g_i \rangle$.
4. **Exit** each step w.p. $0.05 + s \cdot 0.4(1 - \sigma)$; cap $H = 30$; $\sigma_0 = 0.6$.
5. **Knob** $s \in [0, 1]$**:** at $s = 0$ every long-term effect is off, so the myopic policy is optimal (Prop. 2).

Reference policies (`rlrec/sim/policies.py`): random, popular, **myopic oracle** (top-K by *true current* utility, i.e. the ideal supervised model by Lemma 1), and **quality oracle** (top-K by $\hat{r}$, junk excluded). Every policy is no-repeat (it never re-shows clicked items).

Sanity results (`results/sim_sanity.md`, 4,000 users, clicks per session, myopic vs quality):

s	K	myopic	quality	first-slate CTR myopic / quality
0	1	**6.19**	4.95	0.448 / 0.338
0.25	1	3.24	3.29	0.448 / 0.338
0.25	5	5.50	**7.90**	0.769 / 0.698
1	5	2.40	**6.03**	0.769 / 0.698

The myopic oracle always has the higher first-slate CTR; whether it wins long-term depends on s and K. Tests are in `tests/test_user_sim.py`.

S8: Learned agents in the simulator ($K = 1$) (`rlrec/agents/sim_agents.py`, `scripts/run_sim_m1.py`)

Shared network (`RecNet`):

- `StateEncoder`: user embedding, mean of the last clicked items' embeddings, step embedding, and history fill fraction, fed to a 2-layer MLP (64);
- `ItemHead`: score $= \langle Wz + g e_u, e_i \rangle + b_i$;
- embeddings are warm-started from the history ALS.

Protocol: users are split 80/20 (train/test). Logs come from the popularity-softmax/uniform behaviour policy (about 150k / 100k / 75k / 51k transitions at $s = 0$ / 0.25 / 0.5 / 1). Evaluation runs 2 sessions per held-out user in a fresh simulator seed. The metric is **online** (on-policy rollouts in the simulator): first-slate CTR, clicks per session, length, junk clicks.

Agents:

Agent	Training	Acts by	Key settings
Supervised (CE)	MNL choice NLL on logs (`choice_nll`)	top-K of choice head	8 epochs, batch 512, lr 3e-3
SQN-style	CE + double-DQN TD on a Q head (shown item = action)	**choice head**	$\gamma = 0.9$, Polyak 0.01
CQL	double-DQN TD + CQL penalty ($\alpha = 1$)	argmax Q	same
MNL-Q (ours)	CE + TD for Q_c on click transitions + TD for N on no-click transitions	argmax $pQ_c + (1-p)N$	target $y = r + \gamma(1-d)\max_a [pQ_c + (1-p)N]$ from the target net
DQN	**online**, ε-greedy 1→0.05 over 60% of 3,000 steps x 256 users, replay 200k, 2 updates/step, Huber loss, lr 1e-3, Polyak 0.005	argmax Q **within the supervised model's top-20 candidates**	the candidates also restrict the target max
Rating heuristic	none (history ALS)	top-K estimated rating	–
Oracles	true model	see S7	–

MNL-Q in one sentence: show item a, and with probability $p(a \mid s)$ the user clicks (worth $Q_c(s,a)$, which includes the click), otherwise you get the no-click continuation $N(s)$. The click model handles *who clicks*, and the TD heads handle *what a click leads to*. This is Lee & Oh's decomposition $Q(s,A) = \sum P\bar{Q}$ specialised to $K = 1$, learned offline with neural nets.

Results:

Clicks per session, mean ± std over 3 seeds, held-out users (`report/build/midsem_report.md`, section 6):

agent	s = 0	s = 0.25	s = 0.5	s = 1
Popularity	3.05 ± 0.04	2.08 ± 0.01	1.61 ± 0.01	1.18 ± 0.01
Highest estimated rating (heuristic)	4.25 ± 0.05	2.79 ± 0.02	2.20 ± 0.01	1.62 ± 0.02
Supervised (CE)	4.06 ± 0.01	2.47 ± 0.02	1.76 ± 0.02	1.18 ± 0.05
SQN-style (CE + TD aux)	3.85 ± 0.03	2.40 ± 0.02	1.75 ± 0.03	1.20 ± 0.03
Offline RL (CQL)	3.44 ± 0.03	2.22 ± 0.03	1.71 ± 0.01	1.23 ± 0.02
Offline RL (MNL-Q, ours)	3.89 ± 0.00	2.49 ± 0.03	1.85 ± 0.04	1.31 ± 0.05
Online RL (DQN re-ranking top-20)	4.37 ± 0.03	2.46 ± 0.01	1.77 ± 0.02	1.18 ± 0.02
Oracle myopic	6.17 ± 0.07	3.30 ± 0.04	2.25 ± 0.02	1.42 ± 0.03
Oracle quality	4.93 ± 0.06	3.31 ± 0.02	2.67 ± 0.01	2.03 ± 0.04

At $s = 1$: first-slate CTR is 0.299 for supervised vs 0.275 for MNL-Q; junk clicks per session 0.58 vs 0.33; session length 4.6 vs 5.6.

- **MNL-Q vs supervised:** -4% at $s = 0$, $+1\%$ at 0.25, $+5\%$ at 0.5, $+11\%$ at $s = 1$. The advantage appears only when choices change the future and grows with s. At $s = 1$ supervised is no better than popularity (1.18).
- **MNL-Q gives up a little CTR, clicks 43% less junk, and keeps users longer.**
- **Online DQN (top-20)** is best of the learned agents at $s = 0$ (4.37, CTR 0.304), for a short-term reason (better click estimates from interaction). For $s > 0$ it only matches supervised: in 3,000 steps it did not learn the long-term effect.
- **Rating heuristic** beats every learned agent for $s > 0$ (1.62 at $s = 1$); the quality oracle reaches 2.03.

What changed from v1 (`results/sim_m1_v1/`): v1 agents had no history warm start and DQN over all 500 items. Every learned agent was then at or below popularity (supervised 0.82, DQN 0.80 at $s = 1$). That is why we added (i) the warm start, which reflects that real recommenders know the user's past, and (ii) the candidate-restricted DQN, which reflects two-stage recommenders.

S9: Demo (`app/demo.py`, `scripts/train_demo_models.py`)

The Streamlit demo has two pages:

- **Live session:** one simulated user, two recommenders side by side with **the same random seed**, showing clicks, length, first-step click probability, the satisfaction curve, and the shown titles (junk flagged). Strength $s \in \{0, 0.5, 1\}$, with checkpoints for supervised and MNL-Q per strength.
- **Results:** renders the S4/S5/S7/S8 tables and the synthetic figure.

4. Likely cross-questions with answers

Q1. What is novel here? Three things. (i) The framing: we cast supervised and RL recommenders as $\gamma = 0$ vs $\gamma > 0$ policies of the *same* combinatorial MNL MDP. That turns "when does RL win?" into a precise question with two proved bounds: Prop. 1 (myopic can be arbitrarily worse) and Prop. 2 (zero advantage without choice-dependent dynamics). (ii) A controllable simulator whose knob s moves continuously between those two regimes, with slate size as a second axis. (iii) MNL-Q, an offline neural version of Lee & Oh's item-level decomposition. The reproductions themselves are not novel; they are there to validate our tools.

Q2. Isn't your simulator rigged so that RL wins? It is built so that *either* can win, and we show both. At $s = 0$ the theory says supervised is optimal and the data agree: the myopic oracle wins (6.19 vs 4.95), and MNL-Q is slightly *worse* than supervised (3.89 vs 4.06). The long-term mechanisms (satisfaction from item quality, fatigue, clickbait that drives exit) are documented effects in the literature and appear in Lee & Oh's own MovieLens experiment. The knob exists precisely so we report a regime map instead of one cherry-picked setting. What we cannot claim is the *real-world* value of s; that is a limitation.

Q3. Why does a simple rating heuristic beat your RL agent? The heuristic encodes the right long-term prior for free: high predicted rating means high quality, which raises satisfaction and avoids junk. Our simulator rewards exactly that (satisfaction depends on $\hat{r}$). MNL-Q has to *learn* this from about 50k offline transitions collected by a popularity-biased logger, with one-step TD and no exploration, and its value estimates are noisy. The fair comparison is RL vs the supervised model, which is what the topic asks, and there RL wins when s is large. That a domain heuristic wins tells us the RL agent is data- and estimation-limited. Closing that gap (more data, LP slate selection, using rating features in Q) is pre-compre work.

Q4. Why not use KuaiRec? KuaiRec (a fully-observed user-item matrix) would be a better simulator basis, but it was not downloadable from our build environment, so we calibrated on MovieLens-1M, as Lee & Oh did in their real-data experiment. A fully observed matrix removes the need for a rating model; we use ALS predictions instead. The simulator code only needs a $\hat{r}$ matrix, genres, and a junk flag, so switching to KuaiRec is a data-loader change.

Q5. Why only 1 seed and at most 5 epochs on RetailRocket? Compute: a 4-core CPU without a GPU, where one epoch takes 47-75 minutes per model and there are 8 models. The paper's 5 seeds x 50 epochs would be about 2,000 CPU-hours. We use early stopping on validation purchase NDCG@20, and a Colab GPU run is in progress. So the S4 numbers are *partial* and we draw no conclusion yet about SQN vs CE. The point of S4 is to understand Labarca's argument and have the pipeline. Our main claims come from S5-S8, which do not depend on it.

Q6. Your purchase metrics are about 2x the paper's. Is your evaluation broken? We measured why. 67% of test purchase targets are the *same item as the last viewed item* (people view, then add to cart), and 73% are somewhere in the 10-item state; for clicks this is 24% / 37%. So a model that learns "recommend what was just viewed" gets very high purchase HR. The dataset counts match the paper's exactly, so the difference is in event handling: the paper's pipeline likely de-duplicates or treats repeats differently. We reproduce the *protocol* and compare *relative* CE-vs-RL-head differences under one pipeline, not the absolute values.

Q7. What is the difference between your MNL-Q and Lee & Oh's MNL-VQL? They share the structure: item-level values combined through MNL choice probabilities. They differ in five ways.

1. MNL-VQL is **online** with exploration (optimistic utilities and Q bonuses, Eq. 7/9 switching); MNL-Q is **offline**, with point estimates and no exploration.
2. MNL-VQL uses **linear** features with a regret guarantee; MNL-Q uses **neural** networks and has no guarantee.
3. MNL-VQL selects **slates** via the LP/bisection; MNL-Q currently has $K = 1$, so the "assortment" is a single item.
4. MNL-VQL estimates θ by online mirror descent; MNL-Q learns the click model by the MNL log-likelihood on logs.
5. MNL-Q splits the TD loss into click and no-click transitions, Q_c and N.

The neural MNL-Q with LP slate selection ($K > 1$) is the pre-compre step.

Q8. What does γ mean for a recommender? It is how much the system values a future click relative to a click now: $\gamma = 0$ is pure CTR, and $\gamma \to 1$ is total clicks per session. In practice it also sets an effective planning horizon $\approx 1/(1-\gamma)$; we use 0.9, about 10 steps, compared with sessions of 4-16

steps. Labarca's point about SQN's $\gamma = 0.5$ is that it makes Q basically a 2-3 step reward predictor, which is an auxiliary-task signal rather than long-term planning.

Q9. Why did plain DQN fail? DQN over all 500 items had two problems. (i) **Maximisation bias:** the target takes $\max_{a'}$ over 500 noisy Q estimates, which is dominated by the luckiest overestimate. Double DQN reduces this but does not remove it when most items are rarely tried. (ii) **Exploration waste:** ε-greedy over 500 items spends interactions on items that a user would almost never click. That data is uninformative and sessions end quickly. Result: about 1.05 clicks per session at $s = 1$, below supervised. Restricting both acting and the target max to the supervised model's top-20 candidates (the standard two-stage "retrieve then re-rank" design) fixed the variance. It now beats supervised at $s = 0$ (4.37 vs 4.06 over 3 seeds, because online interaction gives better click estimates than the biased logs), but for $s > 0$ it only matches supervised (1.18 vs 1.18 at $s = 1$). Our reading: with 3,000 interaction steps it did not learn the long-term effect, and the top-20 by click probability are full of junk, so the candidate set itself is myopic.

Q10. How do you know the junk items are junk? By construction they are the bottom 25% of the 500-item catalogue by **mean MovieLens rating**, while being in the top 500 by **number of ratings**. They are popular but disliked, the "attractive but low-quality" definition used by Lee & Oh. In the simulator "junk" is a modelling assumption ($\kappa = 1$ extra attractiveness and an exit risk on click), not a learned fact. The agents never see the junk flag; only the oracles do. The heuristic and the agents see only the history ALS. Sensitivity to the 25% threshold and to κ is an ablation we plan.

Q11. What would falsify your hypothesis? The hypothesis is "the RL advantage grows with how choice-dependent the dynamics are (and with slate size)". It would be falsified by: (i) RL beating supervised by a significant margin at $s = 0$, which would contradict Prop. 2 and signal a bug or a supervised baseline that is simply worse at estimation (we check this; the candidate-DQN at $s = 0$ is an example to explain); (ii) the RL-minus-supervised gap *not* increasing in s across seeds; (iii) the quality/myopic gap not growing with K, which would contradict the toy model. We pre-registered (ii) and (iii) through the regime map.

Q12. How does slate size K change the RL advantage? Intuition: a bigger slate makes *some* click likely anyway, so stuffing it with clickbait gains little CTR but still costs exits and satisfaction. Evidence: at $s = 1$ the quality/myopic ratio is 1.4 at $K = 1$ (2.04/1.44) and 2.5 at $K = 5$ (6.03/2.40), and at $s = 0.25$ the two are tied at $K = 1$ (3.24 vs 3.29) but quality wins clearly at $K = 5$ (7.90 vs 5.50). The toy model predicts the ratio grows with K (1.37 to 3.44 from $K = 1$ to $K = 5$ in an illustrative setting). Learned agents at $K > 1$ are pre-compre work.

Q13. Why is the supervised model exactly the $\gamma = 0$ policy? Under what assumptions? Lemma 1: with reward 1 for every click and MNL choice, the probability of a click is $W/(1 + W)$, which increases in the total attraction W, so the best slate is the top-K utilities. A CE model ranks like utilities because softmax is shift-invariant. Assumptions: equal click rewards (with buy = 5 you need the assortment solver), MNL/IIA with no position effects, a correctly specified model with enough capacity and data, no exposure confounding (we condition on the shown slate in `choice_nll`), and the same state information. In practice a *learned* supervised model is only an approximation of the myopic oracle (4.06 vs 6.17 at $s = 0$).

Q14. How did you evaluate, offline vs online? On RetailRocket, offline NIP: full-ranking HR/NDCG on held-out, time-split sessions. That only measures one-step prediction of what the *logging* policy's users did, and by Labarca's Thm 4.3 it cannot reward long-term behaviour. In the simulator, learning is **offline** (from logs) for supervised, SQN, CQL and MNL-Q, and **online** for DQN, but *evaluation* is always **online**: rollouts of each policy on held-out users in a fresh seed, measuring first-slate CTR (short-term) and clicks per session (long-term). That is the advantage of a simulator, and also its weakness (sim-to-real gap).

Q15. In one sentence, what did Labarca et al. show? RL heads improve next-item prediction not because of RL. An auxiliary loss that forces the shared representation to encode session-progress

features (above all the number of interactions so far) does the same job, and a plain regression of that count (HIST) matches SQN.

Q16. Then why use RL at all, if Labarca says it doesn't help? Labarca's result is about NIP metrics on static logs, which are $\gamma = 0$-type metrics. They say so themselves: the theorems apply only under NIP evaluation. Our question is about long-term utility in an environment where actions change the future. There, Prop. 1 shows a myopic policy can be arbitrarily bad, and S5/S7 show large gaps. The two results are complementary: Thm 4.3 is "RL can be arbitrarily worse on NIP" and our Prop. 1 is "supervised can be arbitrarily worse on long-term return".

Q17. What is the difference between SQN and EVAL? SQN learns the *optimal* Q with a max over next actions (control). EVAL learns the Q of the *logging* policy with the actually-logged next item (SARSA-style policy evaluation), so there is no max over unseen items. EVAL performing as well as SQN is evidence that the optimisation part of RL is not what helps NIP.

Q18. Why does the bisection give the exact optimum and not an approximation? $F(A) \geq \lambda$ is equivalent to a linear inequality, so $g(\lambda) = \max_A [\ldots]$ is decreasing with a root exactly at the optimal value. For each λ the inner max is solved exactly by "top-K positive terms". The only approximation is the float precision of λ after 60 halvings, and the selected set is optimal once λ is within the gap between distinct assortment values. Tests confirm it matches brute force and the LP.

Q19. Why does the LP relaxation give integral solutions? For the MNL assortment problem with a cardinality constraint, the relaxed linear-fractional program has an optimal vertex in which every $x_a \in \{0, 1\}$ (Chen & Hausman 2000; Davis et al. 2013). After Charnes-Cooper, optimal y_a equals either 0 or t. The code still thresholds at 0.5 and handles the empty-set case, for numerical safety.

Q20. Why does Myopic fail in the budget MDP? Not buying raises the budget, and at higher budget every purchase pays more. Myopic sets the outside option's value to 0 (`f[:, 0] = 0`) and always tries to sell now. The optimal policy sometimes offers a slate that makes "no purchase" likely (or offers low-index items, which keep the budget up), and so earns more later. It is a clean example of choice-dependent transitions (Prop. 2 does not apply).

Q21. Why is LSVI-UCB so bad? It treats each of 637 (or 21,699) slates as an unrelated action. It must try each slate in each (h, s) before its bonus shrinks, and it cannot share information between slates that contain the same items. MNL-VQL learns a $d = 5$ preference vector and 6-dimensional item values instead. LSVI-UCB is also computationally infeasible at $N = 40$ (760k slates).

Q22. What is the outside option, and why does its value matter? It is "the user picks nothing", with utility 0 by convention (it anchors the MNL scale). In bandits its value is 0, but in an MDP "no click" leads somewhere: a budget increase in S5, a continued session in our simulator (the $N(s)$ head in MNL-Q). If its value is high, showing very attractive items can *hurt*. That is why Lee & Oh need the optimistic/pessimistic utility switch.

Q23. What is CQL doing in your results and why is it conservative? CQL penalises Q-values of actions the logger rarely showed, so it sticks close to logged behaviour (popular items, with a uniform component). It shows few junk items (0.70 at $s = 0$ and 0.13 at $s = 1$, vs 1.91 and 0.58 for supervised) but also has lower CTR (about 0.24 vs 0.29), and it does not win on clicks per session. Offline RL from a weak logger is limited by the logger's coverage.

Q24. Why is your SQN-style agent in the simulator not the same as on RetailRocket? On RetailRocket the action is the clicked item (there is no record of what was shown). In the simulator with $K = 1$ the action is the *shown* item and the reward is click/no-click, which is the correct MDP action. In both cases recommendations come from the CE head, so SQN-style is not an RL policy; it is a supervised policy with an RL auxiliary loss. As expected from Labarca, it behaves like supervised.

Q25. Why warm-start from a history model? Isn't that leaking the simulator? Real recommenders know users' past ratings, so a cold-start agent would be an unrealistic straw man (in v1 all learned agents were at or below popularity). The history ALS is fitted on a random 50% of ratings and is a different model from the simulator's ALS on 100%. So it is informative but not ground truth. The junk flag, fatigue and satisfaction are never given to agents.

Q26. How significant are your differences? One seed? S5 uses 5 seeds and the MNL-VQL vs Myopic gaps (about 25-37%) are far beyond the seed standard deviations (about 0.03-0.08). S7 uses 4,000 users and standard errors of 0.02-0.12. S8 uses 3 seeds. The seed standard deviations are 0.01-0.05 clicks, and the MNL-Q vs supervised gap at $s = 1$ (1.31 vs 1.18, +11%) is only about 2-3 such standard deviations, so we call it a consistent trend across s, not a large effect. The −4% / +1% / +5% / +11% ordering is the evidence. For the final report we will add standard errors and paired comparisons on the same users and seeds.

Q27. Your MNL-Q is worse than supervised at $s = 0$. Doesn't that hurt your story? No, it is what theory predicts. At $s = 0$ the myopic policy is optimal (Prop. 2), so any $\gamma > 0$ agent can only match supervised and, with noisier TD targets, slightly underperform it (3.89 vs 4.06, −4%). The cost of using RL when it is not needed is small, and the benefit when it is needed is real. That trade-off is the content of the regime map.

Q28. Why $\gamma = 0.9$ in the agents but undiscounted clicks in evaluation? $\gamma < 1$ stabilises TD learning (bounded targets, faster contraction). With exits and $H = 30$ the effective horizon of 10 steps covers most sessions. The evaluation metric is the quantity we care about. A γ sweep is in the regime map.

Q29. What is the state your agents see? Isn't the true state hidden? They see user id, the last 10 clicked items, and the step index. Satisfaction and fatigue are hidden, so the problem is a POMDP. The click history is a sufficient-ish proxy: fatigue is a deterministic function of the click history, and satisfaction is a function of the history plus the user's ratings. That is why the oracles, which see true utilities, are upper references, not achievable targets.

Q30. Why is the first-slate CTR of the myopic oracle always higher, and why report it? By Lemma 1 it maximises the immediate click probability, so no policy can beat it on that metric. We report it to show the short-term vs long-term trade-off explicitly: higher CTR now (0.769 vs 0.698 at $K = 5$) can come with 2.5x fewer clicks per session. This is exactly the failure an A/B test on CTR alone would miss.

Q31. Couldn't you just add a junk penalty to the supervised model and be done? If you know what junk is, yes. That is the rating/quality heuristic, and it works. The point of RL is to discover *which* choices hurt future engagement from interaction data, without hand-labelling them. In real systems the damaging mechanism (fatigue, clickbait, filter bubbles) is not known in advance. The heuristic is a strong baseline that we report honestly.

Q32. What is double DQN, briefly? Use the online network to pick the best next action and the target network to evaluate it: $y = r + \gamma Q_{\theta'}(s', \arg\max_a Q_\theta(s', a))$. This decouples the noise that picks the max from the noise that values it, which reduces overestimation.

Q33. What does PISDR add, and why is it your next baseline? PISDR (Yuan et al., ACML 2024, Alibaba) treats *re-ranking* as offline RL at two levels, page sequence and item sequence within a page. It uses a decision-transformer-style sequence model to reduce extrapolation error, and a multi-tower critic for several feedback signals (CTR, conversion). It reports about 1.4% AUC/MAP/NDCG gains on Avito and +2.59% CTR in an online A/B test. It is the industrial counterpart of our $K > 1$ extension; a "PISDR-lite" (sequence-model policy plus critic, offline) is our planned offline-RL re-ranking baseline.

Q34. How do you ensure the code does what the formulation says? There are unit tests: solvers vs brute force; choice probabilities sum to 1 and ignore empty slots; $s = 0$ makes the dynamics choice-independent; myopic wins CTR and quality wins return with strong effects; and reproducibility with seeds. In S5 the optimal policy is computed by exact DP with the true model, and MNL-VQL converging to it (99.7-100%) is an end-to-end check of the solver and the decomposition.

Q35. What are HR@20 and NDCG@20 measuring, and why both? HR@20 asks whether the true next item is in the top 20. NDCG@20 also rewards *where* it is, $1/\log_2(\text{rank}+2)$. We evaluate with full ranking over all 70k items (no sampled negatives, which are known to distort comparisons), separately for clicks and purchases.

Q36. Why a time-based split? A random split lets the model train on sessions from the future of the test sessions, which leaks popularity trends (Hidasi & Czapp 2023). Labarca use 80/10/10 by session start time, and so do we.

5. Honest limitations, and what pre-compre will add

Limitations (say them before they are asked).

1. **Simulator dependence.** Long-term effects are modelled, not measured, and the "real" value of s is unknown.
2. **The learned RL agent does not yet beat a domain heuristic** (rating heuristic 1.62 vs MNL-Q 1.31 at $s = 1$), and it is far from the quality oracle (2.03). The MNL-Q gain over supervised is modest (+11% at $s = 1$).
3. **Learned agents only at** $K = 1$ so far; the slate results ($K = 5$) are oracle-only.
4. **S4 is partial:** 1 seed, 2-3 epochs on CPU; SQN currently below CE; absolute numbers inflated by repeat purchases.
5. **S8 has only 3 seeds**; differences at $s \le 0.5$ are within a few standard deviations.
6. MNL-VQL used tuned bonus constants and $\bar{\sigma} = 1$, so no formal guarantee applies to our runs exactly.
7. MNL assumes IIA and no position bias; real slates have position effects.
8. The two-type slate toy model is a conjecture with numerical support, not a proof.

Pre-compre plan.

1. **Regime map** over $K \in \{1, 3, 5, 10\}$ x strength $s \in \{0, 0.25, 0.5, 1\}$ x $\gamma \in \{0, 0.5, 0.9, 0.99\}$: the RL-minus-supervised gap in clicks per session, with first-slate CTR alongside.
2. **Neural MNL-Q re-ranker with LP/bisection slate selection** for $K > 1$: item-level Q_c, N and the choice model plug straight into `solve_parametric` (values $f_a = Q_c(s, a)$, $f_0 = N(s)$, weights e^v).
3. **PISDR-lite** as an offline-RL re-ranking baseline.
4. **More seeds** (at least 5) with paired standard errors, and the GPU run of S4 with EVAL/HIST rows.
5. **Ablations:** junk fraction and κ; the warm start; candidate-set size for DQN; the logging policy's coverage (ε); the MNL-Q click/no-click split vs plain Q.
6. Analyse the toy slate model properly, and possibly prove monotonicity in K under conditions.

One-line closing answer ("so when does RL beat supervised?"): when the user's choice changes their future (satisfaction, fatigue, exit) strongly enough, and more so with larger slates. When it doesn't, supervised is already optimal, and RL can only cost you a little.